

Documentation Technique — SP1 Client léger PHP

Castellane-Auto · Site web PHP / MySQL BTS SIO option SLAM — Session 2026 Équipe vlndkaizen — Mouhammad El Bachir DIENG · Djihane Haroune · Youssouf Kadi · Maria Benadjila · Massamba Gad

1. Vue d'ensemble du projet

Le projet **Castellane-Auto** se compose de **deux applications** complémentaires partageant la même base de données MySQL. Ce document concerne uniquement la **Situation Professionnelle n°1 (SP1)** : le site web (client léger).

Élément	Détail
Application	Site web Castellane-Auto
Type	Client léger
Technologies	HTML5 / CSS3 / PHP 8 / MySQL
Public cible	Visiteurs, candidats, moniteurs, administrateur
Hébergement dev	WampServer (Apache + PHP 8 + MySQL 8)
Dépôt Git	https://github.com/vlndkaizen/E6_Client_leger.git

 **Lien avec la SP2** : le site et l'application desktop Java partagent la **même base** `auto_ecole` . Les deux applications appliquent strictement le patron **MVC**. Voir le document `05_DocTech_SP2_Java.pdf` pour le client lourd.

2. Architecture MVC

2.1 Arborescence du projet

```

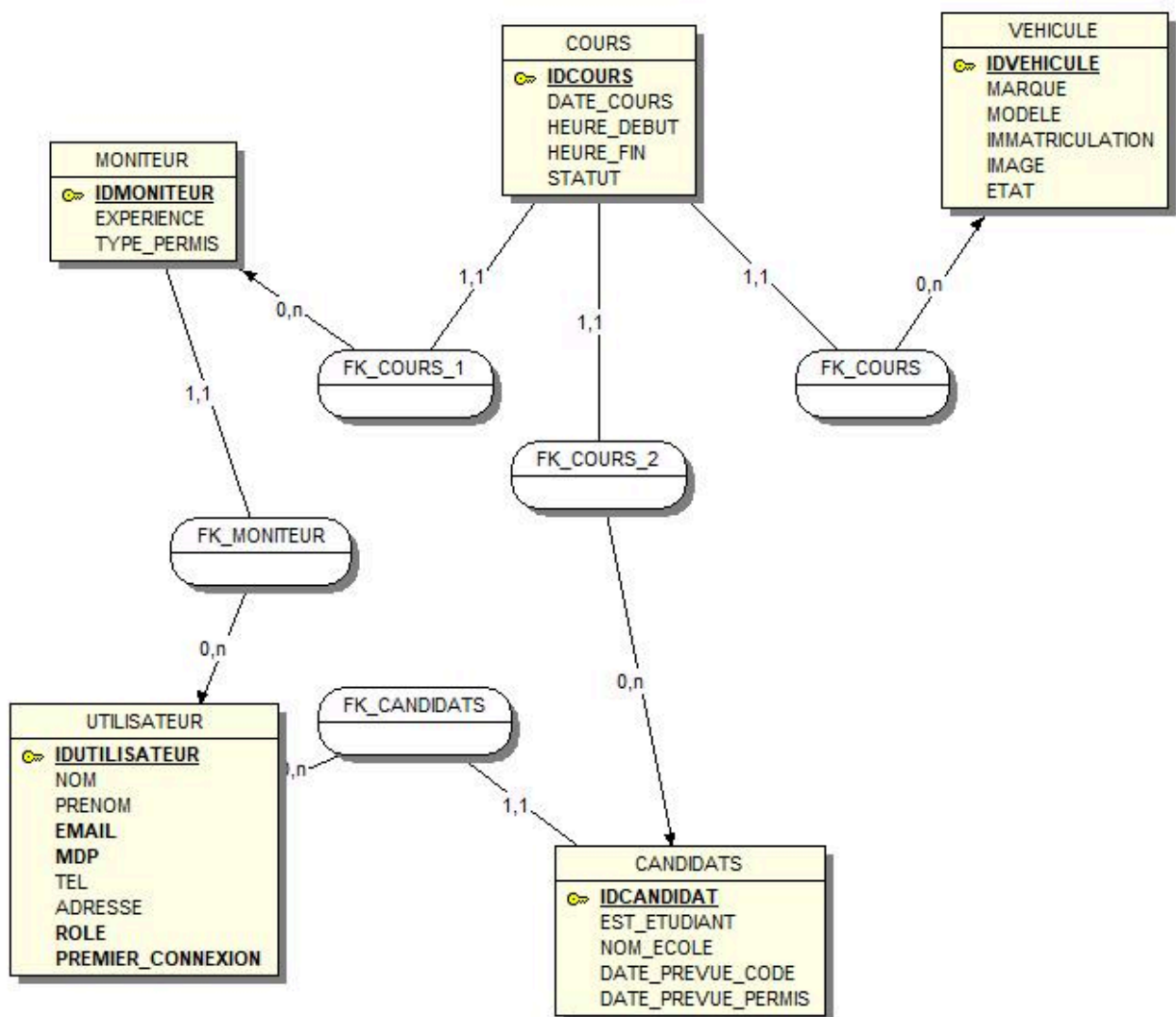
castellane_auto/
├── index.php                ← Point d'entrée + routeur
├── controleur/
│   └── controleur.class.php ← Logique métier + validation
├── modele/
│   └── modele.class.php     ← Accès BDD (PDO)
├── vue/
│   ├── entete.php          ← En-tête commun
│   ├── public/             ← Pages publiques
│   ├── admin/              ← Pages admin
│   ├── candidat/           ← Pages candidat
│   ├── moniteur/           ← Pages moniteur
│   └── vue_connexion.php
├── sql/
│   └── auto_ecole.sql       ← Script BDD
├── design/                 ← CSS, JavaScript
├── image/                  ← Logos, fond
└── uploads/                ← Images véhicules

```

2.2 Flux d'une requête type

1. L'utilisateur fait une requête HTTP vers `index.php?page=candidats`
2. Le **routeur** (`index.php`) identifie la page demandée et vérifie les droits (session + rôle)
3. Le **contrôleur** (`controleur.class.php`) reçoit les données du formulaire et les valide
4. Le contrôleur appelle le **modèle** (`modele.class.php`) qui exécute la requête SQL via PDO
5. Le résultat est passé à la **vue** appropriée (`vue/admin/candidats.php`) qui produit le HTML
6. Le HTML est renvoyé au navigateur

3. Modèle Conceptuel des Données (MCD)



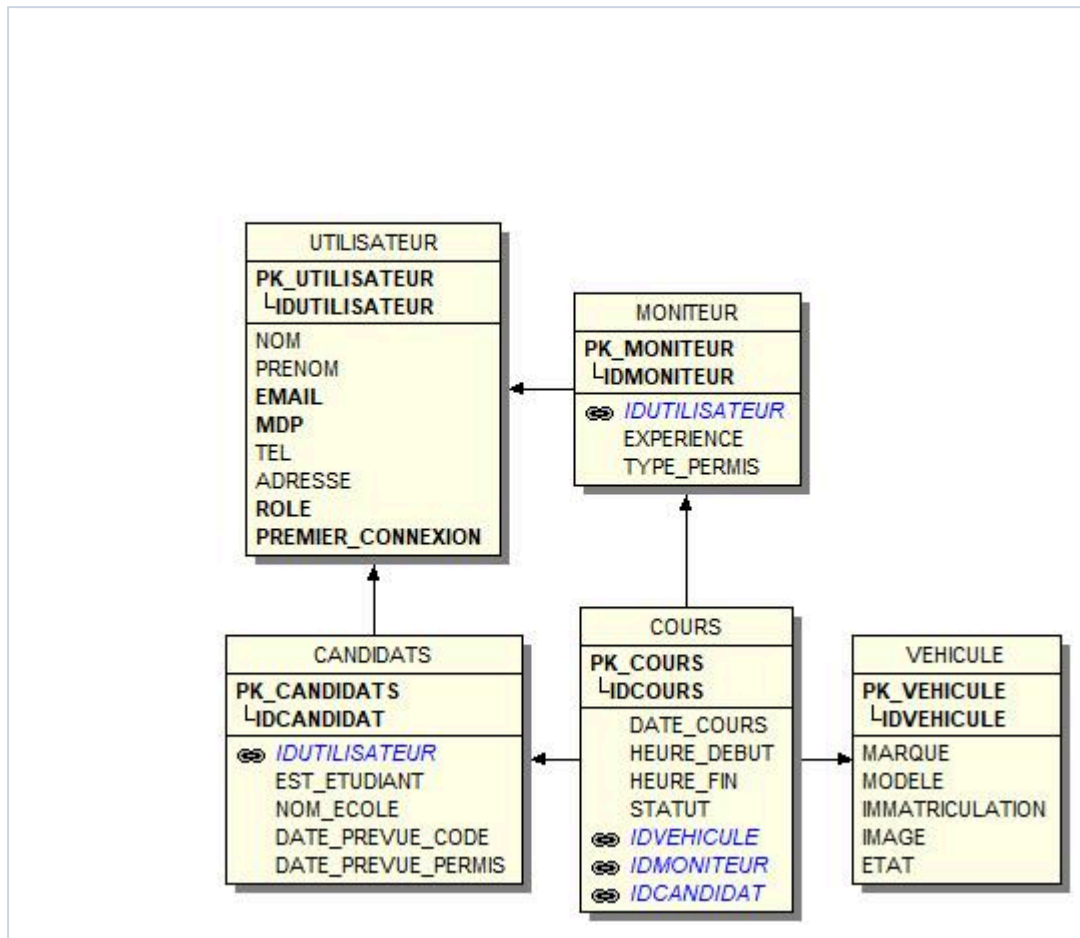
Entité	Rôle
UTILISATEUR	Entité mère contenant les attributs communs à tous les utilisateurs
CANDIDATS	Spécialise UTILISATEUR pour les élèves de l'auto-école
MONITEUR	Spécialise UTILISATEUR pour les moniteurs
VEHICULE	Représente les voitures et motos du parc
COURS	Représente une leçon de conduite planifiée

Associations

- **FK_CANDIDATS** (CANDIDATS → UTILISATEUR) : héritage, 1,1 ↔ 0,n
- **FK_MONITEUR** (MONITEUR → UTILISATEUR) : héritage, 1,1 ↔ 0,n
- **FK_COURS_1** (COURS → MONITEUR) : un cours est assuré par un moniteur, 1,1 ↔ 0,n
- **FK_COURS_2** (COURS → CANDIDATS) : un cours est suivi par un candidat, 1,1 ↔ 0,n

- **FK_COURS** (COURS → VEHICULE) : un cours utilise un véhicule, 1,1 ↔ 0,n

4. Modèle Logique des Données (MLD)



```

utilisateur (idutilisateur, nom, prenom, email, mdp, tel, adresse, role, premier_connexion)
  clé primaire : idutilisateur
  contrainte : email UNIQUE
  contrainte : role ENUM('admin', 'candidat', 'moniteur')
  
```

```

candidats (idcandidat, #idutilisateur, est_etudiant, nom_ecole,
           date_prevue_code, date_prevue_permis)
  clé primaire : idcandidat
  clé étrangère : idutilisateur → utilisateur(idutilisateur) ON DELETE CASCADE
  
```

```

moniteur (idmoniteur, #idutilisateur, experience, type_permis)
  clé primaire : idmoniteur
  clé étrangère : idutilisateur → utilisateur(idutilisateur) ON DELETE CASCADE
  
```

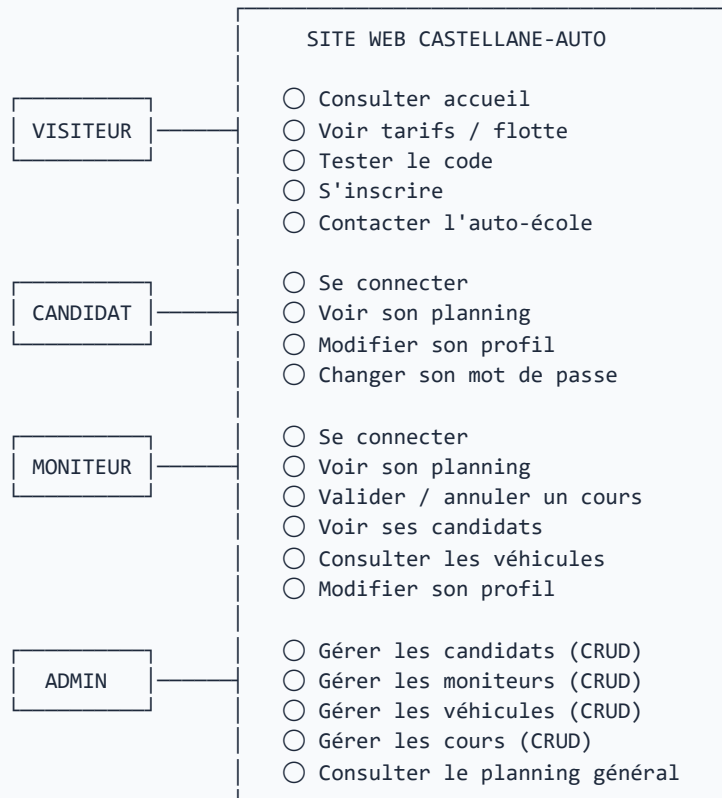
```

vehicule (idvehicule, marque, modele, immatriculation, image, etat)
  clé primaire : idvehicule
  
```

```

cours (idcours, date_cours, heure_debut, heure_fin, statut,
       #idvehicule, #idmoniteur, #idcandidat)
  clé primaire : idcours
  clé étrangère : idvehicule → vehicule(idvehicule) ON DELETE CASCADE
  clé étrangère : idmoniteur → moniteur(idmoniteur) ON DELETE CASCADE
  clé étrangère : idcandidat → candidats(idcandidat) ON DELETE CASCADE
  
```

5. Diagramme de Cas d'Usage (DCU) du site web



6. Maquettes du site web

6.1 Front-office (visiteur)

Page	Contenu
Accueil	Bannière + présentation auto-école + boutons "Voir les tarifs", "S'inscrire" (visiteur uniquement)
Tarifs	Tableau des formations (Permis B, A, BSR, conduite accompagnée) avec prix
Flotte	Galerie photos des véhicules (Peugeot 207, motos roadster)
Test du code	QCM aléatoire de 10 questions
Inscription	Formulaire (nom, prénom, email, mot de passe, date naissance)
Contact	Adresse, téléphone, e-mail, formulaire de contact

6.2 Back-office (admin)

Page	Contenu
Tableau de bord admin	Stats (nb candidats, nb cours à venir, nb véhicules dispo)
Gestion candidats	Tableau + boutons Ajouter/Modifier/Supprimer
Gestion moniteurs	Idem
Gestion véhicules	Idem + upload d'image
Gestion cours	Calendrier + ajout de cours avec détection conflit horaire

6.3 Espace candidat / moniteur

Page	Contenu
Mon planning	Liste des cours du candidat / moniteur connecté
Mon profil	Lecture seule + bouton "Modifier" (toggle JS)
Mes candidats (moniteur)	Liste avec stats par candidat
Véhicules (moniteur)	Vue lecture seule avec état coloré

7. Choix techniques justifiés

7.1 Pourquoi PHP ?

- **Gratuité** : pas de coût de licence (open source)
- **Compatibilité** : fonctionne avec tous les hébergeurs standards
- **Richesse de l'écosystème** : sessions, sécurité, BDD natives
- **Mature** : utilisé par >75% des sites web dans le monde
- **Bonne intégration MySQL** via PDO

7.2 Pourquoi MySQL ?

- **Gratuit** (édition Community)
- **Performances** adaptées à la volumétrie de l'auto-école
- **Connecteurs** matures (PDO pour PHP)
- **Outils d'administration** matures (phpMyAdmin)

7.3 Pourquoi MVC ?

- **Séparation des responsabilités** : modèle (données), vue (affichage), contrôleur (logique)
- **Maintenabilité** : modification d'une partie sans casser les autres
- **Réutilisabilité** : le même contrôleur peut servir plusieurs vues
- **Testabilité** : chaque couche peut être testée indépendamment
- **Standard de l'industrie** : connu et attendu par tous les développeurs

8. Sécurité applicative

Mesure	Implémentation	Protection contre
Hashage MDP (v2)	<code>password_hash(\$mdp, PASSWORD_BCRYPT)</code>	Vol de BDD
Vérification MDP (v2)	<code>password_verify(\$saisie, \$hash)</code>	Comparaison sécurisée
Requêtes préparées	<code>PDO prepare() + bindValue()</code>	Injection SQL
Validation regex	Sur nom, prénom, e-mail, tél, MDP	Données malformées
Échappement HTML	<code>htmlspecialchars()</code> sur toutes les sorties	XSS
Upload sécurisé	<code>finfo</code> (vérif MIME réel) + nom de fichier aléatoire	Upload de scripts malveillants
Gestion des rôles	<code>\$_SESSION['role']</code> (lue depuis la BDD, plus de hardcoding)	Élévation de privilèges
Déconnexion en POST	Formulaire POST au lieu de GET <code>?page=9</code>	CSRF
Forçage changement MDP	Flag <code>premier_connexion</code> en BDD	MDP par défaut
Détection conflit horaire	<code>verifierConflitHoraire()</code>	Double-booking (RG2)
Contrôle état véhicule	<code>vehiculeEstDisponible()</code>	Cours avec voiture en réparation (RG6)

9. Environnement de développement et de production

Élément	Développement	Production cible
OS développement	Windows 10/11	—
OS cible	—	Tous OS (navigateur web)
Serveur web	Apache (WampServer)	Apache mutualisé
SGBD	MySQL 8 (WampServer)	MySQL 8 mutualisé
IDE	Visual Studio Code	—
Versioning	Git / GitHub	—
Navigateurs supportés	Chrome, Firefox, Edge, Safari	Idem

10. Accès aux livrables

Ressource	Lien
Code source SP1 (ce projet)	https://github.com/vIndkaizen/E6_Client_leger.git
Code source SP2 (client lourd)	https://github.com/vIndkaizen/E6_Client_Lourd.git
Drive Google (documentation complète)	https://tinyurl.com/2p9v79dd

11. Évolutions récentes (client léger PHP)

11.1 Sécurité et authentification

Évolution	Détail
Rôle issu de la BDD	<code>\$_SESSION['role']</code> lit la valeur <code>role</code> de la table <code>utilisateur</code> (plus de hardcoding)
Déconnexion en POST	Migration de <code>?page=9</code> (GET) vers un formulaire POST <code><button name="logout"></code> → protection anti-CSRF basique
idutilisateur en session	Permet à un candidat / moniteur connecté d'identifier son propre profil pour la modification

11.2 Nouvelles fonctionnalités candidat

- **Modification du profil** (page `?page=53`)
- Affichage en mode **lecture** par défaut
- Bouton « Modifier mes informations » qui révèle le formulaire (toggle JS)
- Validation côté serveur (email valide, etc.)
- Le MDP n'est mis à jour QUE si un nouveau est saisi (champ vide = conservation de l'ancien)

11.3 Nouvelles fonctionnalités moniteur

Fonctionnalité	Page	Description
Mes Candidats	<code>?page=61</code>	Liste des candidats encadrés, avec stats : nb cours à venir, nb effectués, total
Véhicules	<code>?page=62</code>	Vue lecture seule de toute la flotte avec leur état coloré (Disponible / En réparation / Indisponible)
Mon Profil	<code>?page=63</code>	Mode lecture + bouton Modifier
Validation cours	Sur planning	Boutons « Valider » / « Annuler » sur chaque cours <code>À venir</code>

11.4 Règles de gestion ajoutées (côté PHP)

Code	Règle
RG6	Impossibilité de planifier un cours avec un véhicule dont l'état <code>≠ Disponible</code> . Vérifié dans <code>Modele::insert_cours()</code> et <code>update_cours()</code> .
RG7	Un moniteur ne peut modifier que ses propres cours (sécurité par <code>WHERE idmoniteur = :idmoniteur</code>).
RG8	Un moniteur ne peut affecter qu' <code>Effectué</code> ou <code>Annulé</code> à un cours (whitelist côté serveur). Pas de retour à <code>À venir</code> .

11.5 Bugs résolus

Voir le document `08_Incidents_SP1_PHP.pdf` pour le détail complet : 11 incidents tracés sur le client léger PHP (10 résolus + 1 planifié v2).